AURIX

Microcontroller Infineon Software Architecture

User's Manual

IRQ Driver

Release V2.6

2014-08

Microcontrollers

Edition 2014-08

Published by
Infineon Technologies AG
81726 Munich, Germany

LEGAL DISCLAIMER

THE INFORMATION GIVEN IN THIS DOCUMENT IS GIVEN AS A HINT FOR THE IMPLEMENTATION OF THE INFINEON TECHNOLOGIES COMPONENT ONLY AND SHALL NOT BE REGARDED AS ANY DESCRIPTION OR WARRANTY OF A CERTAIN FUNCTIONALITY, CONDITION OR QUALITY OF THE INFINEON TECHNOLOGIES COMPONENT. THE RECIPIENT OF THIS DOCUMENT MUST VERIFY ANY FUNCTION DESCRIBED HEREIN IN THE REAL APPLICATION. INFINEON TECHNOLOGIES HEREBY DISCLAIMS ANY AND ALL WARRANTIES AND LIABILITIES OF ANY KIND (INCLUDING WITHOUT LIMITATION WARRANTIES OF NON-INFRINGEMENT OF INTELLECTUAL PROPERTY RIGHTS OF ANY THIRD PARTY) WITH RESPECT TO ANY AND ALL INFORMATION GIVEN IN THIS DOCUMENT.

Information

For further information on technology, delivery terms and conditions and prices, please contact the nearest Infineon Technologies Office (**www.infineon.com**).

Warnings

Due to technical requirements, components may contain dangerous substances. For information on the types in question, please contact the nearest Infineon Technologies Office.

Infineon Technologies components may be used in life-support devices or systems only with the express written approval of Infineon Technologies, if a failure of such components can reasonably be expected to cause the failure of that life-support device or system or to affect the safety or effectiveness of that device or system. Life support devices or systems are intended to be implanted in the human body or to support and/or maintain and sustain and/or protect human life. If they fail, it is reasonable to assume that the health of the user or other persons may be endangered.

Date	Version	Change Description
2014-08-05	2.6	HW managed interrupt vector table supported. Inttab and Irq_InstallInterruptHandler functions are removed.
2014-07-15	2.5	TC27xC and TC23x usermanual reference added
2013-11-22	2.4	Configuration parameters added with SFR's modified information
2013-09-24	2.3	Updated the section 6.2.2 for Interrupt Frame Functions name change
2013-08-26	2.2	Updated the TC29x reference
2013-06-24	2.1	Updated the TC26x reference
2013-04-04	2.0	CommonPublishedInformation container info added
2013-01-30	1.0	Interrupt Vector Table information updated
2013-01-03	0.3	Syntax of Interrupt Frame Functions updated
2012-05-16	0.2	Updated for TC27xA step
2011-03-31	0.1	Initial Version

We Listen to Your Comments

Is there any information in this document that you feel is wrong, unclear or missing?
Your feedback will help us to continuously improve the quality of this document.
Please send your proposal (including a reference to this document) to:

mcdocu.comments@infineon.com

Table of Contents **Page**

List of Figures **Page**

List of Tables **Page**

List of Code Listings **Page**

1 Introduction

This User Manual provides information regarding the functionality, configuration parameters and API implementation for the IRQ Driver.

This User Manual is intended to help the user to get acquainted with the IRQ Driver implementation for the AURIX Hardware Platform. It is the documentation that describes how to use the IRQ Driver.

1.1 Scope

This document addresses the following features of the IRQ Driver implementation,

- File structure of the Driver
- Configuration parameters of the Driver
- Parameters published by the Driver
- API Specification
- Common use cases with code snippets.
- Limitations and Assumptions

This document addresses the driver implementation for the following variants of Aurix Family

Some of the peripheral units/feature might not be available in all the variants. In such cases, the relevant information should be discarded for the variant where the unit/feature is not applicable.

1.2 Abbreviations

The abbreviations used inside this document are explained in **Table 1**.

Table 1 Abbreviations used in this document

Abbreviation	Explanation
µC	Microcontroller
µs	Microsecond
API	Application Programming Interface
AUTOSAR	AUTomotive Open System Architecture
BSW	Basic Software
CPU	Central Processing Unit
DET	Development Error Tracer
IFX	Infineon Technologies
IRQ	Interrupt Request Module
MCAL	MicroController Abstraction Layer
MC-ISAR	Microcontroller Infineon Software Architecture
RAM	Random Access Memory
ROM	Read Only Memory
SCU	System Control Unit
SFR	Special Function Register
SRN	Service Request Node
SW	Software
TBD	To Be Defined

1.3 References

This section lists down all relevant documents for this User Manual.

Refer to the latest version for the documents without version information.

[1] Target Data Sheet: tc27x_um_V1.3.1.pdf, tc27xC_um_v1.5.pdf, tc26xB_um_v1.2.pdf, tc29xB_um_v1.2_2.pdf, tc23x_tc22x_um_v1.0.pdf

[2] Header file for Irq: Irq.h

2 Driver Overview

This driver provides the necessary configuration parameters and API for interrupt configuration and initialization and handling. IRQ module and all configurations in this module are only applicable if no OSEK-OS or Autosar OS is used, which typically takes care to configure the interrupts.

2.1 Software Hardware Mapping

This section introduces the user to the HW features used for implementation. The **Figure 1** illustrates the Hardware peripherals and their interaction with the Software Drivers.

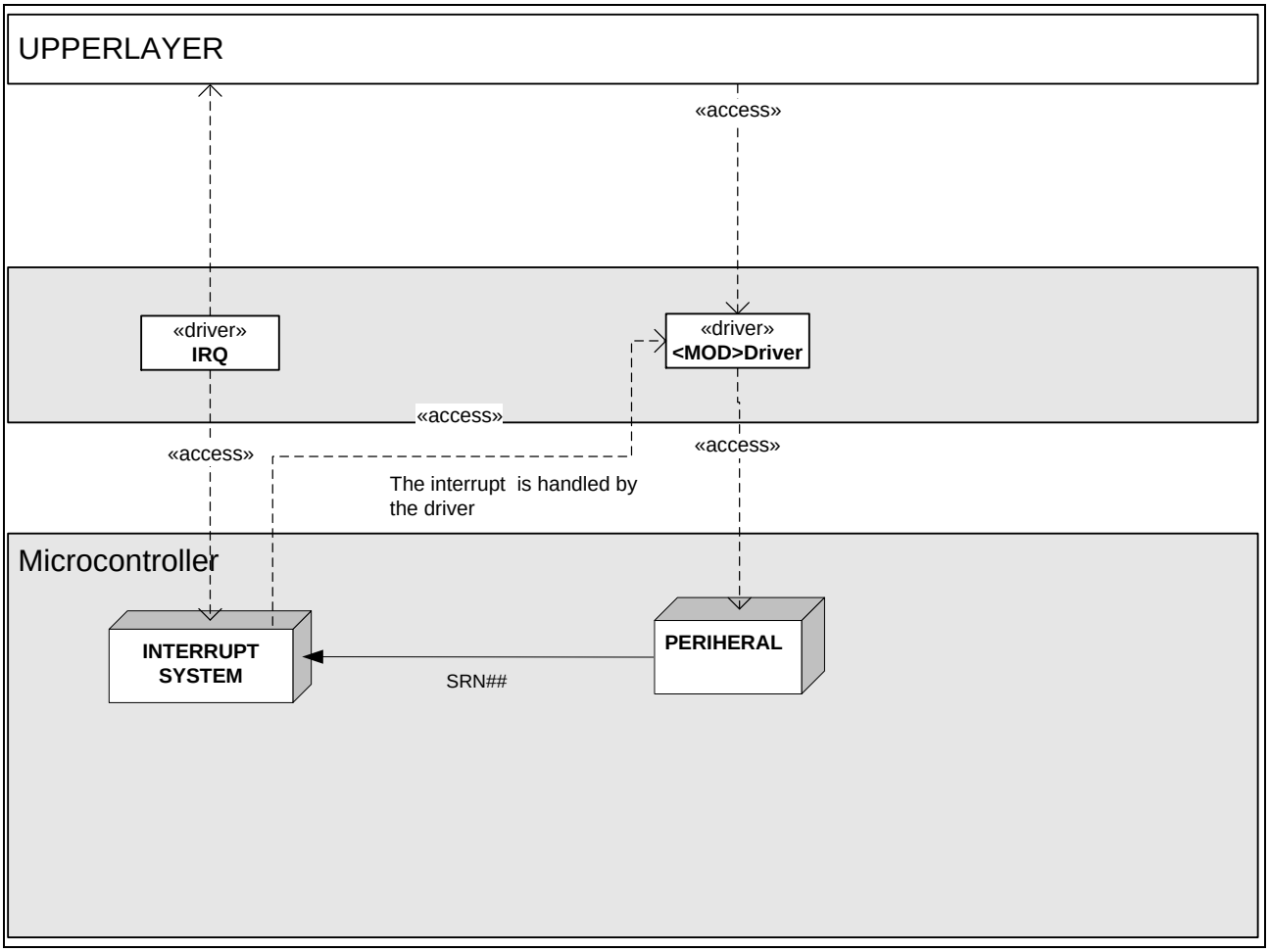

Figure 1 SW HW Mapping

The IRQ driver accesses the Interrupt System and the Peripherals. It initializes the peripheral specific interrupt prioirity. Upon an interrupt, the interrupt system jumps to the appropriate interrupt handler frame within the IRQ driver. From the interrupt frame the corresponding interrupt handler function is called. In the

MCAL drivers the interrupt handler functions are located within the SW driver. In the case of complex driver the application may contain the interrupt handler function.

2.2 Software Driver Description

The **Figure 2** shows the Application Programming Interface (API) for the <modulename> Driver and its interaction with other software Drivers.

The APIs of the Driver are explained in detail in chapter **6**

The Configuration Data of the Driver is also depicted in the figure. The individual containers and their configuration parameters are explained in detail in chapter **4**.

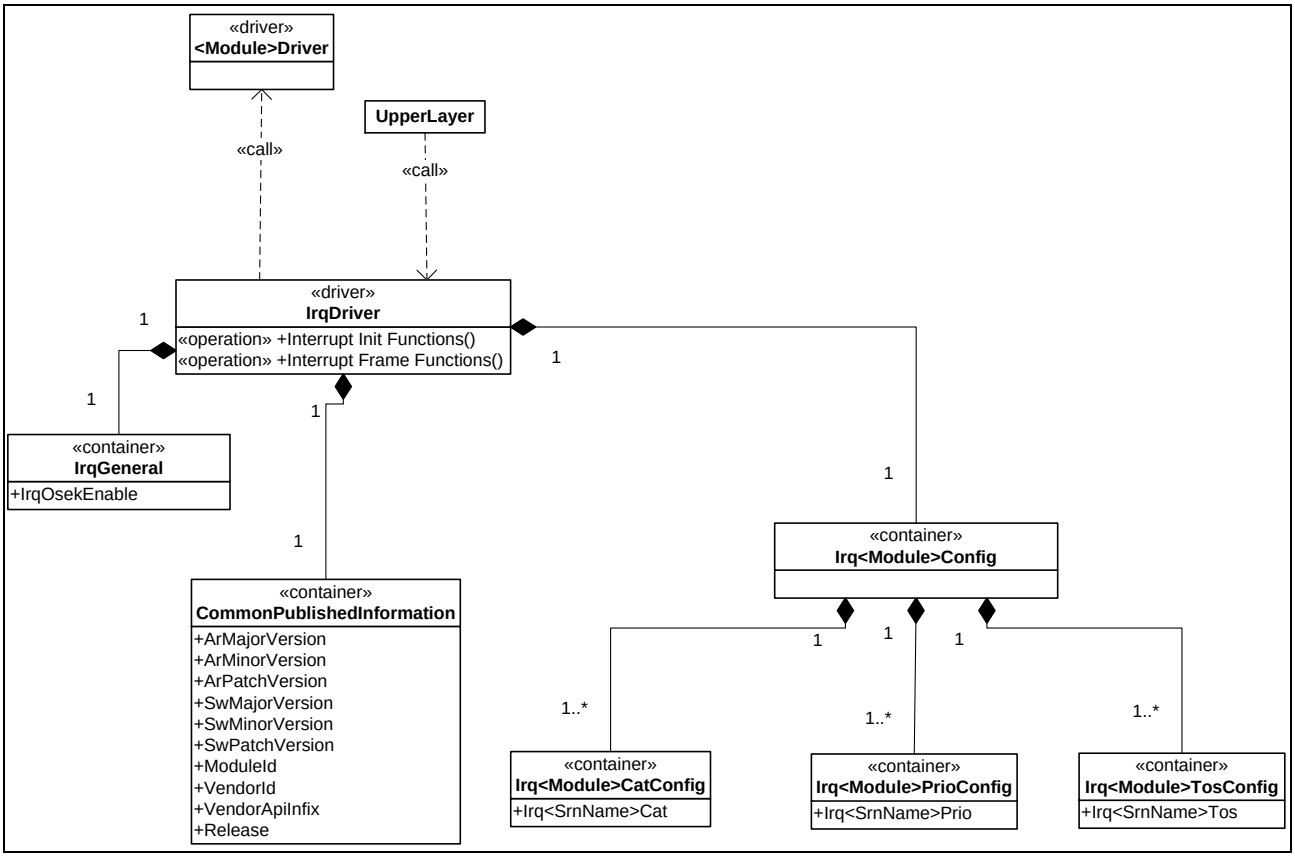

Figure 2 IRQ Driver Description

3 File structure

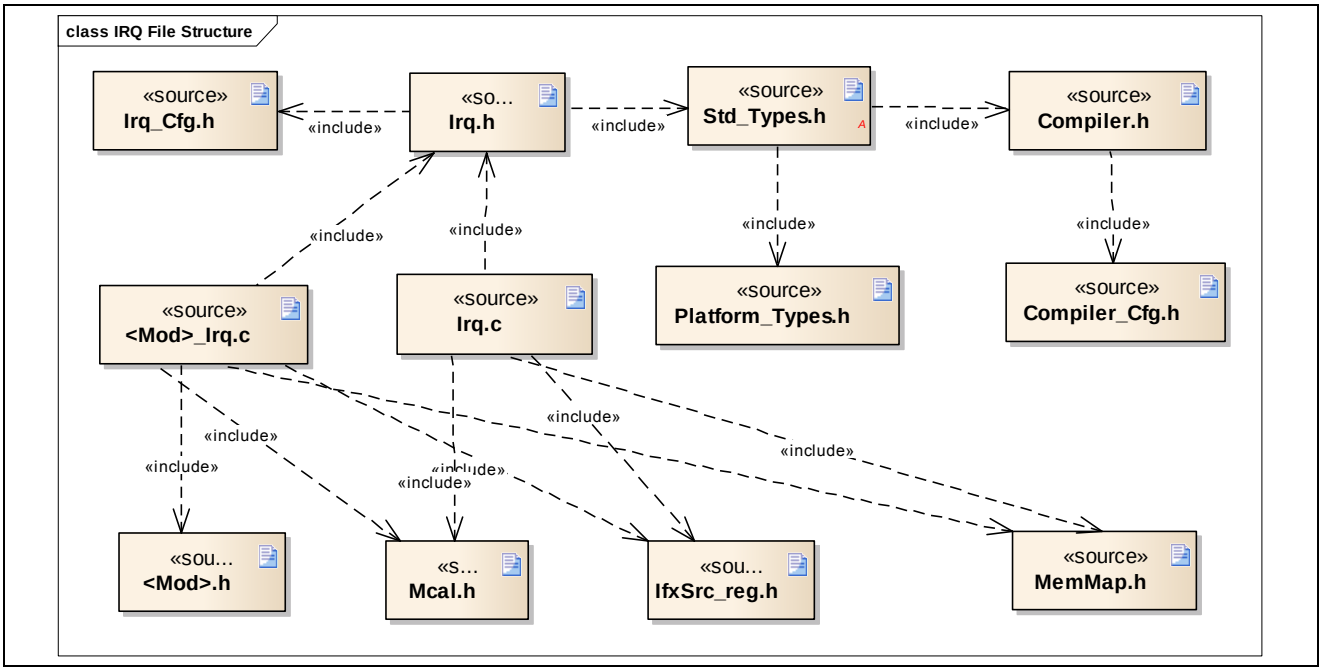

Figure 3 Driver File Structure

Table 2 File Contents

File Name	File Contents
Platform_Types.h	Platform Dependencies (AURIX) are declared here.
Compiler.h	Compiler Dependencies are declared here.
Compiler_Cfg.h	Contains abstraction of compiler specific keywords used for addressing data and code.
Std_Types.h	Standard Data types to be used are declared here.
Irq.h	Interrupt Priority Initialization functions are exported.
Irq.c	Interrupt Priority Initialization functions are implemented.
Irq_Cfg.h	Configuration Data of Interrupt Priority and Interrupt Category.
<Mod>_Irq.c	Module specific Interrupt Frames are implemented. From these frames the respective module interrupt handlers are called.
<Mod>.h	The module specific interrupt handlers are exported.
IfxSrc_reg.h	Includes the Src register definition file for AURIX microcontroller
Mcal.h	Provides library functions for MCAL layer
Det.h	Provides the functionality of Development Error Tracer
MemMap.h	Mapping of code and data (variables, constant variables) to specific memory sections

The files Platform_Types.h, Compiler.h, Std_Types.h and MemMap.h are provided by AUTOSAR and are used as it is without any changes.

4 Configuration Documentation

This chapter summarizes

- Configuration concept used for structuring the Driver

- All configuration parameters and their description

4.1 Configuration Concept

4.1.1 Configuration Class

The development of Basic Software Drivers involves the following development cycles:

- Compiling,
- Linking and
- Downloading of the executable to ECU memory.

Configuration of parameters can be done in any of these process-steps: pre-compile time, link time or even post-build time. According to the process-step that does the configuration of parameters, the configuration classes are categorized as below

Pre-compile time: The pre-compile time configuration is implemented by means of compiler switches or variables affecting the total build process. Pre-compiler configuration requires the Driver to be compiled, linked and located and hence the values of pre-compile parameters must be **frozen before compilation** and therefore can be used in source code distributions of the Driver only.

Link time: The Link time configuration doesn't affect compilation stage but impacts the linkage and locating stages. The Link time parameters can be used in binary (library) distributions of the Driver and the values of the Link time parameters are specified by means of external configuration source code that must be linked to the Driver during the build process.

Post-build time: The Post-build time configuration will not affect the build process and hence Post-build time configuration is applied to Driver at runtime. Since configuration location is statically independent of the Driver functionality itself but linked dynamically (e.g. at Driver initialization), post-build time configuration is used for re-programmable configuration data.

4.1.2 Configuration Variant

Based on the most suitable configuration class that may be needed for the configuration parameters of the Driver, a Driver can be delivered as the following variants

Variant PC: This variant is limited to pre-compile configuration parameters only. The intention of this variant is to optimize the parameters configuration for a source code delivery.

Variant LT: This variant allows a mix of pre-compile time- link time configuration parameters. The intention of this variant is to optimize the parameters configuration for an object code delivery.

Variant PB: This variant allows a mix of pre-compile time-, post build-time configuration parameters. The intention of this variant is to optimize the configuration parameters for a re-loadable binary.

The IRQ Driver is delivered as a Variant PC.

4.1.3 How to read the Configuration Class field

The examples cited below help the user to identify the configuration class of the parameter for a given delivery type.

*Note: Topics **4.1.1**, **4.1.2** should be read for understanding the configuration class field*

Table 3 Configuration Class Example 1

Configuration Class:	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--

'x' indicates that the parameter is implemented as a pre-compile parameter in all types of deliveries i.e. Variant PB, Variant LT, Variant PC.

Table 4 Configuration Class Example 2

Configuration Class:	Pre-compile	x	Variant PC
	Link Time	--	--
	Post build	x	Variant PB

'x' indicates that the parameter is implemented as a

1. pre-compile parameter if the delivery type is Variant PC delivery
2. post-build parameter if the delivery type is a Variant PB delivery

4.2 Container: IrqGeneral

Interrupt Module global settings and module specific settings are present in this container.

4.2.1 IrqOSekEnable

Table 5 IrqOSekEnable

Syntax:	IrqOSekEnable		
Source:	This parameter is microcontroller specific		
File:	Irq_Cfg.h		
Range:	TRUE: OSEK OS is used. This allows the user to configure the Interrupt category as CAT1 or CAT23 FALSE: OSEK OS is not used. All Interrupt must be of category CAT1.		
Default Value:	FALSE		
Configuration class	Pre-compile	x	All Variants
	Link Time		
	Post build		
Description:	Parameter to identify the use of OS. This controls the available configuration options for Interrupt Category.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	None		

4.3 Container: Irq<ModuleName>Config

This container contains the module specific interrupt configuration parameters.

4.3.1 Container: Irq<ModuleName>CatConfig

4.3.1.1 Irq<SrnName>Cat

Syntax:	Irq<SrnName>Cat
Source:	This parameter is microcontroller specific.
File:	Irq_Cfg.h
Range:	CAT1

	CAT23		
Default value:	CAT1		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	The interrupt category setting of the corresponding SRN of the Module. The SRN number indicates the SRN configured. Depending upon the category the interrupt frame is different.		
SFRs Modified:	None		
Scope:	Driver		
Dependency:	Only the option CAT1 is allowed to select, If the parameter IrqOSekEnable is set to FALSE.		

4.3.2 Container: Irq<ModuleName>PrioConfig

This container contains the parameter to configure the priority of all the SRN supported by the module.

4.3.2.1 Irq<SrnName>Prio

Table 6 Irq<SrnName>Prio

Syntax:	Irq<SrnName>Prio		
Source:	This parameter is microcontroller specific.		
File:	Irq_Cfg.h		
Range:	0 to 255		
Default Value:	0		
Configuration class	Pre-compile	x	All Variants
	Link Time	--	--
	Post build	--	--
Description:	The interrupt priority setting of the corresponding SRN of the Module. The SRN number indicates the SRN configured.		
SFRs Modified:	SRC_<SrnName>		
Scope:	Driver		
Dependency:	None		

4.3.3 Container: Irq<ModuleName>TosConfig

This container contains the parameter to configure the type of service of all the SRN supported by the module.

4.3.3.1 Irq<SrnName>Tos

Syntax:	Irq<SrnName>Tos
Source:	This parameter is microcontroller specific.
File:	Irq_Cfg.h
Range:	CPU0 CPU1 CPU2

	DMA SDMA		
Default value:	CPU0		
Configuration class	Pre-compile	X	All Variants
	Link Time	--	--
	Post build	--	--
Description:	The type of service setting of the corresponding SRN of the Module. This defines the Interrupt Control Unit which service the ISR.		
SFRs Modified:	SRC_<SrnName>		
Scope:	Driver		
Dependency:	None		

4.4 Container: CommonPublishedInformation

This container contains the common published information. These parameters are non editable.

4.4.1 ArMajorVersion

Table 7 ArMajorVersion

Name:	ArMajorVersion
Source:	This parameter is defined by IFX
File:	NA
Range:	0 - 255
Default value:	0
Description:	Major version number of AUTOSAR specification on which the appropri ate implementation is based on.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.2 ArMinorVersion

Table 8 ArMinorVersion

Name:	ArMinorVersion
Source:	This parameter is defined by IFX
File:	NA
Range:	0 - 255
Default value:	0
Description:	Minor version number of AUTOSAR specification on which the appropri ate implementation is based on.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.3 ArPatchVersion

Table 9 ArPatchVersion

Name:	ArPatchVersion
Source:	This parameter is defined by IFX
File:	NA
Range:	0 - 255
Default value:	0
Description:	Patch version number of AUTOSAR specification on which the appropr iate implementation is based on.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.4 SwMajorVersion

Table 10 SwMajorVersion

Name:	SwMajorVersion
Source:	This parameter is defined by IFX
File:	Irq.xdm
Range:	0 - 255
Default value:	3
Description:	Major version number of the vendor specific implementation of the Driver.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.5 SwMinorVersion

Table 11 SwMinorVersion

Name:	SwMinorVersion
Source:	This parameter is defined by IFX
File:	Irq.xdm
Range:	0 - 255
Default value:	0
Description:	Minor version number of the vendor specific implementation of the Driver.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.6 SwPatchVersion

Table 12 SwPatchVersion

Name:	SwPatchVersion
Source:	This parameter is defined by IFX
File:	Irq.xdm
Range:	0 - 255
Default value:	0
Description:	Patch version number of the vendor specific implementation of the Driver.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.7 ModuleId

Table 13 ModuleId

Name:	ModuleId
Source:	This parameter is defined by IFX
File:	Irq_Cfg.h
Default value:	255
Description:	IRQ Driver module.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.8 VendorId

Table 14 VendorId

Name:	VendorId
Source:	This parameter is defined by IFX
File:	Irq_Cfg.h
Default value:	17
Description:	Vendor ID of the dedicated implementation of this Driver according to the AUTOSAR vendor list. Vendor is Infineon Technologies.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.4.9 VendorApiInfix

Table 15 VendorApiInfix

Name:	VendorApiInfix
Source:	This parameter is defined by IFX

Name:	VendorApiInfix
File:	Irq.xdm
Default value:	None
Description:	Vendor Specific Name used for extending the API names along with VendorId.
SFRs Modified:	None
Scope:	Not Applicable for ADC driver
Dependency:	None

4.4.10 Release

Table 16 Release

Name:	Release
Source:	This parameter is defined by IFX
File:	Adc.xdm
Range:	_TRICORE_TC299 _TRICORE_TC298 _TRICORE_TC277 _TRICORE_TC275 _TRICORE_TC267 _TRICORE_TC265 _TRICORE_TC264 _TRICORE_TC233 _TRICORE_TC234 _TRICORE_TC237
Description:	Aurix derivative used for implementation.
SFRs Modified:	None
Scope:	Driver
Dependency:	None

4.5 Derived Configuration Parameters

There are no derived parameters in this driver.

5 Published parameters

This section describes the parameters published by the MCU Driver.

5.1 VendorId

Table 17 VendorId

Syntax:	IRQ_VENDOR_ID
Type:	Macro - #define
File:	Irq_Cfg.h
Value:	17

Description:	Vendor is Infineon Technologies

5.2 ModuleId

Table 18 ModuleId

Syntax:	IRQ_MODULE_ID
Type:	Macro - #define
File:	Irq_Cfg.h
Value:	255
Description:	This macro gives the IRQ Driver module ID.

6 API Documentation

6.1 API Type Definitions

There are no type definitions in this module.

6.2 API Functions

This chapter describes the API interfaces provided by IRQ Driver.

6.2.1 Interrupt Init Functions

Table 19 Irq<ModuleName>_Init

Service name:	Irq<ModuleName>_Init
Syntax:	void Irq<ModuleName>_Init (void)
Service ID:	None
Sync/ Async:	Synchronous
Reentrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	Initializes the Module Interrupt Prioirty settings. The function is located in the file Irq.c
Caveats:	None
Configuration:	Irq<SrnName>Prio
DET:	None
DEM:	None
Implementation Comments:	The service initializes the priority and the type of service of the available modules SRN. Update the Interrupt vector table with the address of corresponding Interrupt Frame Functions.

6.2.2 Interrupt Frame Functions

Table 20 Interrupt Frame Functions

Service name:	Interrupt Frame Functions
Syntax:	IFX_INTERRUPT(<SRNNAME>_ISR, 0, <SRNPRIO>) OR ISR(<SRNNAME>_ISR)
Service ID:	None
Sync/ Async:	Synchronous
Reentrancy:	Non-reentrant
Parameters (in):	None
Parameters (out):	None
Return value:	None
Description:	• The respective Module interrupt hanlers (located in <ModuleName>.c) are called from interrupt Frame functions. • The function is located in the file <ModuleName>_Irq.c • The syntax of the function depends on the Interrupt category (Irq<SrnName>Cat) configured for the respective SRN##. If the category is CAT1 the syntax used is: IFX_INTERRUPT(<SRNNAME>_ISR, 0, <SRNPRIO>) If the category is CAT23 the syntax used is: ISR(<SRNNAME>_ISR) • In case of complex drivers the user is allowed to write the user interrupt handler in this function.
Caveats:	None
Configuration:	Irq<SrnName>Cat
DET:	None
DEM:	None
Implementation Comments:	None

7 Error Classification

NA

8 Example Usage

8.1 Configuration of Driver

Follow the steps below to configure the IRQ Module

- Irq General **[4.2]**.
  - Configure IrqOsekEnable to indicate the use of OS
  - Configure IrqNoOfArbitrationCycle to indicate the arbitration cycle to be used. Note that this parameter determines the available interrupt priority options.
  - Configure IrqNoOfClksPerArbCycle depending upon the System Clock Frequency used.
- Module Interrupt Settings **[4.3]**.
  - Configure Irq<ModuleName>SR##Prio to indicate the priority of the corresponsing SRN
  - Configure Irq<ModuleName>SR##Cat to indicate the category of the corresponsing SRN

8.2 Initialization of Interrupt Priority of the Driver

An example initialization of the Interrupt of the Driver is depicted in the **Code Listing 1**

Code Listing 1 <modulename> Interrupt Initialization

```
001:   /* MCU Initialization*/
002:   Mcu_Init(&(Mcu_ConfigRoot[0]));
003:
004:   /* MCU Clock / System Clock Initialization Clk: 60Mhz. */
005:   Mcu_InitClock(0);
006:   while(Mcu_GetPllStatus(0) == 0);
007:   Mcu_DistributePllClock(0);
008:
009:   /* Configure Module Interrupt Priority */
010:   Irq<ModuleName>_Init
011:   ..
012:   ..
013:   /* Enable Global Interrupt Flag. */
014:   EnableAllInterrupts();
015:
016:   /* Initialization of Module */
017:    <ModuleName>_Init(ConfigPtr);
018:   ..
019:   ..
```

9 Assumptions and Limitations

9.1 Assumptions and Deviations from the Software Specification

NA

9.2 Limitations

NA

9.3 Hardware Features Not Supported

NA

w w w . i n f i n e o n . c o m

Published by Infineon Technologies AG